

Taylorred Expressions Inventory Manager

Security & Bug Audit Report

Report date: 2026-05-02 | Branch: **master-UI-Rework** | App version: **1.0.2**

Executive summary

An automated audit of the JandHInventory WPF/.NET 8 codebase identified **1 critical**, **4 high**, **9 medium**, and **10 low**-severity findings spanning credentials hygiene, stock-tracking logic, schema bootstrap, file-system handling, and CSV export. Six findings have been fixed in this pass; the remainder are documented below with concrete remediation steps. The single critical finding (committed plaintext database credentials) requires user action — passwords must be rotated and the file untracked from version control by a human, not by an automated tool.

Severity	Count	Fixed in this pass	Requires user action
Critical	1	0	1
High	4	3	1
Medium	9	2	0
Low	10	1	0
Total	24	6	2

Methodology

An automated agent enumerated the codebase (Services, ViewModels, Views, Data, Models, App startup, configuration files) and inspected each suspect file in full. Categories audited: credential exposure, SQL injection, path traversal, unsafe deserialization, WebView2 navigation, OCR temp-file handling, Process.Start argument flow, DbContext lifetime, async/threading correctness, resource disposal, exception swallowing, ObservableCollection thread safety, manual schema migration safety, and stock/percentage logic. Generated CommunityToolkit.Mvvm partials and the bin/obj/packages directories were excluded.

Fixes applied in this pass

#	Title	Location	Fix
1	Stock level logic returned wrong label	Services/InventoryService.cs:1976	GetStockLevel compared stock to goodAt for the Low check; when goodAt was unset (-1), every above-zero stock fell through to the default 'Low' return. Replaced with an explicit ladder using lowAt as the upper bound for Low.

2	Purchase delete left stock & price stale	Services/InventoryService.cs:1693	DeleteItemPurchaseAsync removed the row but never reversed the stock add or recomputed the item's latest-purchase price/date — symmetric inverse of AddItemPurchaseAsync. Now subtracts (quantity × packSize) for tracked types and rebinds Price/DatePurchased to the next-most-recent remaining purchase.
3	Schema bootstrap re-ran per InventoryService	Services/InventoryService.cs:10	Each new InventoryService() ran EnsureCreated() plus eight EnsureXxxTable DDL probes. Multiple ViewModels create their own service instances. Added a static lock + bool gate so the bootstrap runs at most once per process; the intentional offline-mode swallow is preserved.
4	Thumbnail invalidation used wrong glob	Services/ThumbnailCacheService.cs:289	Disk cache writes .jpg files but InvalidateItem only deleted .png matches, so updated images stayed stale on disk. Glob now matches all extensions for the item id.
5	CSV export vulnerable to formula injection	Services/ExportService.cs:169	CsvEscape quoted commas/quotes/newlines but did not neutralize Excel formula prefixes (=, +, -, @, tab, CR). A user-entered item name like =cmd 'c calc!A0 would execute when the CSV was opened in Excel. Now prefixes such cells with a single quote so they render as text.
6	First() on potentially empty list	Services/InventoryService.cs:1635	AddItemPurchaseAsync called .First() on the purchases list to find the latest. The just-added row is normally there, but a concurrent delete could race it away and throw inside the using block. Switched to FirstOrDefault() with a fallback to the local purchase variable.

Detailed findings

Critical

C1. Database credentials checked into version control

Critical

User action

Location: appsettings.json (tracked in git)

What's wrong: The local SQL Server password (User Id=JandH; Password=Meeko2021!) and the read-only Azure catalog DB password (tebuddy_readonly with a complex password) are in plaintext in a tracked file. Anyone with repository access — past or future contributors, anyone who clones a fork — has these credentials, including in git history.

Fix: 1) Rotate both passwords immediately. 2) Untrack the file: `git rm --cached appsettings.json` and add it to .gitignore. 3) Commit a sanitized appsettings.example.json. 4) Load real credentials from a per-machine file outside the repo, Windows Credential Manager, or DPAPI. Rewriting git history to scrub the secret is optional but recommended; assume both passwords are compromised regardless.

High

H1. GetStockLevel ignored configured LowThreshold (FIXED)

High

Fixed

Location: Services/InventoryService.cs:1976

What's wrong: The Low check was `stock > outAt && stock < goodAt`. When the Good threshold was unset (-1), every non-Out stock fell through to the default 'Low' return — even high stock. Cards on the home dashboard misreported status.

Fix: Replaced with an explicit ladder: Out if `stock ≤ outAt`, Low if `lowAt ≥ 0` and `stock ≤ lowAt`, Good if `goodAt ≥ 0` and `stock ≥ goodAt`, otherwise Low.

H2. DeleteItemPurchaseAsync left stock and price stale (FIXED)

High

Fixed

Location: Services/InventoryService.cs:1693

What's wrong: AddItemPurchaseAsync writes the item's latest-purchase price/date and adds (quantity × packSize) to CurrentStock for tracked types. Delete removed the purchase row only — stock stayed inflated and Price/DatePurchased still pointed at the deleted record.

Fix: Now mirrors AddItemPurchaseAsync: subtracts the deleted purchase's stock contribution (clamped to 0), then re-binds Price and DatePurchased to the next-most-recent remaining purchase (or leaves them if none remains).

H3. Schema bootstrap ran on every InventoryService construction (FIXED)

High

Fixed

Location: Services/InventoryService.cs:10

What's wrong: Multiple ViewModels new up their own InventoryService. The constructor invoked EnsureCreated() and eight EnsureXxxTable DDL probes per instance, adding launch latency and racing concurrent app launches against the IF NOT EXISTS / CREATE TABLE blocks (SQL Server allows concurrent execution and a duplicate-object error can leak).

Fix: Wrapped the bootstrap in a static lock + bool gate. The intentional offline-mode try/catch (a deliberate UX choice for the DB-offline banner) is preserved.

H4. EnsureXxxTable swallowed every exception silently

High

Not fixed

Location: Services/InventoryService.cs:55, 75, 110, 132, 158, 180, 209, 232

What's wrong: Each schema-bootstrap helper logs and returns on any exception, so a partial schema mismatch (e.g. a column rename that didn't run) is invisible at runtime and surfaces as cryptic insert/read failures hours later.

Fix: Not auto-fixed because the project intentionally favors offline resilience — rethrowing here could break the app's launch path on transient SQL outages. Recommend: distinguish 'connection failed' (continue, banner) from 'DDL failed because object exists / column conflict' (rethrow). Easiest path: switch from manual EnsureXxxTable helpers to EF Core migrations.

Medium

M1. UpdateService launches installer without signature verification

Medium

Not fixed

Location: Services/UpdateService.cs:122

What's wrong: The installer is loaded from a constrained UNC share (validated at lines 102-107), but version.txt and the chosen installer file are writable by anyone with SMB write on the share. A user clicking 'Update' would launch whatever sits there — potentially elevated.

Fix: Verify the installer's Authenticode signature against a pinned publisher cert before launching. WinVerifyTrust via P/Invoke or X509Certificate.CreateFromSignedFile, then check thumbprint.

M2. DbContext created before semaphore in EnsureXxxOnceAsync

Medium

Not fixed

Location: Services/InventoryService.cs:2206-2210 (and similar)

What's wrong: The using-var-context line runs before await _tableEnsureLock.WaitAsync(), so when many concurrent callers race the lock, multiple contexts are created and held while waiting. Minor leak, no functional bug today.

Fix: Move CreateContext() inside the try block after the await on the semaphore.

M3. Tesseract engine instantiated per OCR call

Medium

Not fixed

Location: Services/SentimentService.cs:462

What's wrong: Each call constructs and disposes a new TesseractEngine, which loads the trained-data file from disk every time. Already noted in a prior internal audit; still open. Slow extraction perceived by user.

Fix: Cache one TesseractEngine per language as a static lazily-initialized field, dispose on App.OnExit. Engines are not thread-safe — guard with a lock or use a per-thread / pooled instance.

M4. DetectBorderedRegions uses Bitmap.GetPixel in nested loops

Medium

Not fixed

Location: Services/SentimentService.cs:586-633

What's wrong: GetPixel locks/unlocks GDI bits per call; the 5-pixel scan plus FloodFillFindBounds drives this into the millions of locks for a 1200-px image. Visible UI stall during sentiment crop.

Fix: Replace with a single LockBits + Marshal.Copy into a byte[], read pixels by index, UnlockBits when done.

M5. ThumbnailCacheService.Invalidateltem used wrong glob (FIXED)

Medium

Fixed

Location: Services/ThumbnailCacheService.cs:289

What's wrong: Disk cache writes .jpg (DiskCachePath builds {itemId}_{hash}.jpg) but invalidation only matched .png, so updated images persisted stale on disk after edits.

Fix: Glob now matches {itemId}_* so all variants and any legacy .png entries are cleared.

M6. ExportService.CsvEscape vulnerable to CSV / formula injection (FIXED)

Medium

Fixed

Location: Services/ExportService.cs:169

What's wrong: Cells starting with =, +, -, @, tab, or CR are treated as formulas by Excel. A user setting an item name to =cmd|'/c calc '!A0 would execute when the export was opened in Excel.

Fix: Now prefixes any such cell with a single quote so Excel renders it as text. Quote/comma/newline escaping is preserved.

M7. Plaintext credentials embedded in process-wide static

Medium

User action

Location: Data/InventoryDbContext.cs:32

What's wrong: The connection string is cached for 5 minutes inside the DbContext. The locking is correct, but the cached value is read in OnConfiguring on arbitrary threads. The deeper issue is that the credentials are plaintext in the source file — see C1.

Fix: Resolve via C1 (rotate, untrack, load from external store). Once the source no longer carries the secret, the static cache is fine.

M8. Static-bool guards duplicated across many EnsureOnceAsync helpers

Medium

Not fixed

Location: Services/InventoryService.cs (multiple)

What's wrong: Several helpers use a per-method static bool + SemaphoreSlim. The pattern is correct but duplicated; future maintenance is more error-prone than it needs to be.

Fix: Extract to a single helper: EnsureOnceAsync(ref bool flag, SemaphoreSlim sem, Func<Task> init).

M9. async void in non-event-handler ViewModel methods

Medium

Not fixed

Location: ViewModels/InspirationViewModel.cs:541, 1107, 1754

What's wrong: ApplyFiltersAsync, LoadSelectableItemsForAdd, NavigateToItem are async void but invoked directly (e.g. from partial void OnSearchTextChanged), not as event handlers. An unhandled exception inside crashes the process via TaskScheduler_UnobservedTaskException.

Fix: Convert each to async Task and have callers await (or fire-and-forget with a top-level try/catch that logs). Skipped here because each call site needs individual review.

Low

L1. Server and database name written to network log

Low

Not fixed

Location: Data/InventoryDbContext.cs:104

What's wrong: LogInfo records server and database names on a network share readable by other users. No credentials, but useful to an attacker for reconnaissance.

Fix: Either drop the server/db from the message, or write a redacted form (e.g. hostname hash).

L2. Catalog-lookup IN-list built by string concatenation

Low

Not fixed

Location: Services/CatalogLookupService.cs:145

What's wrong: imgSql interpolates string.Join(',', results.Select(r => r.Id)) into the query. Today r.Id is int so it's safe — fragile if the type ever changes.

Fix: Parameterize via STRING_SPLIT or build a named parameter set.

L3. Single-process logging serializes through one global lock

Low

Not fixed

Location: Services/LoggingService.cs:53

What's wrong: File.AppendAllText opens and closes the log file on every line under one global LockObject; functional but measurably slow under heavy logging.

Fix: Buffer to a ConcurrentQueue and flush every N ms or N entries on a worker.

L4. User settings deserialized without options constraints

Low

Not fixed

Location: Services/UserSettingsService.cs:54

What's wrong: JsonSerializer.Deserialize(json) accepts arbitrary JSON. Settings file is per-user under %LOCALAPPDATA%, so attacker has user-level access already — low impact.

Fix: Pass a JsonSerializerOptions with strict number handling and explicit type discriminators if the model ever grows polymorphic.

L5. N+1 query in AddProjectCreationAsync

Low

Not fixed

Location: Services/InventoryService.cs:2050

What's wrong: var item = await context.Items.FindAsync(itemId) inside a foreach for each material. For projects with many materials, this is one round-trip per material.

Fix: Batch-load with Where(i => ids.Contains(i.Id)) before the loop, dictionary by Id.

L6. First() on potentially empty list (FIXED)

Low

Fixed

Location: Services/InventoryService.cs:1635

What's wrong: AddItemPurchaseAsync called .First() on the purchases list. Normally the just-added row is there, but a concurrent delete race could throw inside the using block.

Fix: Replaced with FirstOrDefault() ?? purchase so the just-added row is the safe fallback.

L7. Culture-sensitive ToTitleCase

Low

Not fixed

Location: Services/SentimentService.cs:902

What's wrong: ToTitleCase uses CurrentCulture; in tr-TR/az locales 'i'/'I' would normalize differently. Low impact on Windows-only en-US deployments.

Fix: Pass CultureInfo.InvariantCulture.TextInfo for sentiment normalization.

L8. HttpClient AllowAutoRedirect on TE wishlist import

Low

Not fixed

Location: Services/TEWishlistImportService.cs:35

What's wrong: User-supplied wishlist URL with auto-redirect could be 302'd to internal hosts. No auth flows through the client so impact is minimal; no SSRF surface.

Fix: Disable auto-redirect and validate the response URL before fetching the next hop.

L9. BitmapImage stream lifetime

Low

Not fixed

Location: Views/SentimentCropView.xaml.cs:122; Views/ItemLookupDialog.xaml.cs:136

What's wrong: BitmapImage with StreamSource = stream inside a using block could outlive the stream. Today CacheOption=OnLoad is set, so it's safe — flagged because the pattern is brittle if someone later removes OnLoad.

Fix: Add a code comment noting the OnLoad dependency, or buffer the bytes into a MemoryStream that is not disposed prematurely.

L10. Hard-coded UNC config and log paths

Low

Not fixed

Location: Services/ConfigPathService.cs:8 (and LoggingService, UpdateService)

What's wrong: All shared resources point at \\Win-u5iq2hisnh3\\e\... — a single-machine deployment. If the host is renamed or replaced, every client needs a rebuild.

Fix: Move the share host and root to appsettings.json (with the per-machine file from C1) so deployments are portable.

Recommendations & next steps

In priority order:

Today: Rotate the JandH SQL Server password and the tebuddy_readonly Azure password (C1 / M7). Treat both as compromised.

Today: Untrack appsettings.json (`git rm --cached`), add to .gitignore, ship appsettings.example.json. Optionally rewrite history to scrub the leaked secrets.

This week: Pin Authenticode publisher cert verification on the auto-update path (M1).

This week: Cache a single TesseractEngine instance and switch DetectBorderedRegions to LockBits (M3, M4) — biggest perceived-perf wins.

This sprint: Convert the manual EnsureXxxTable helpers to EF Core migrations and let H4's silent swallow disappear naturally.

This sprint: Convert the three async void ViewModel methods (M9) to async Task and audit their call sites.

Backlog: Address the L-tier hardening items as code is touched in those areas; none are urgent.

All fixes applied in this pass were verified by a successful `dotnet build JandHInventory.csproj` with 0 errors.